

МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ЛЬВІВСЬКА ПОЛІТЕХНІКА



АВТОМАТИЗОВАНЕ ПРОЕКТУВАННЯ
КОМП'ЮТЕРНИХ СИСТЕМ

Лабораторна робота № 5

«Реалізація автоматизованих тестів»

Виконав:

КІ-404

Рибка С.О.

Прийняв: Федак П.Р.

Львів - 2024

Завдання

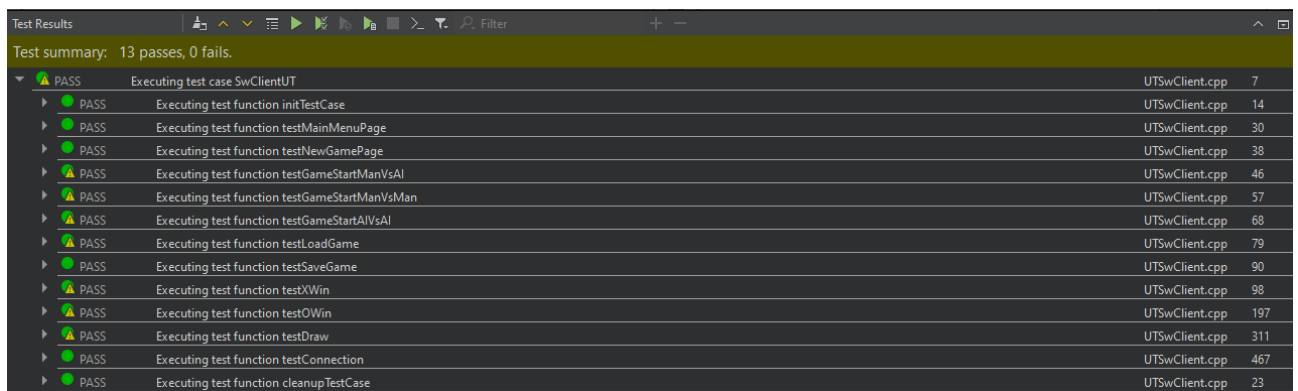
1. Реалізувати або використати існуючий фреймворк для тестування.
2. Створити набір автоматизованих тестів.
3. Звіт про тестування повинен містити кількість усіх тестів, складених тестів, нескладених тестів, покриття.
4. Покриття повинно бути більше 80%.

Згідно із моїм номером у списку маю таке завдання:

Student number	Game	Config format
17	Tic-Tac-Toe 3x3	JSON

Виконання

Оскільки у проєкті використовую фреймворк Qt, то для тестування використав Qt Test. За допомогою Qt Test можна зручно тестувати сигнали та слоти, які є основним засобом комунікації між об'єктами класів у Qt. Також перевагою Qt Test є інтеграція із Qt Creator (див. рис. 1) – середовищем розробки для програм, що використовують вищезгаданий фреймворк.



The screenshot shows the 'Test Results' window in Qt Creator. The title bar says 'Test Results'. Below the title bar, there's a summary: 'Test summary: 13 passes, 0 fails.' The main area is a table with columns for test status, test name, and file/line number. All tests are marked as 'PASS' with a green circle icon. The tests are listed as follows:

Test Status	Test Name	File/Line
PASS	Executing test case SwClientUT	UTSwClient.cpp 7
PASS	Executing test function initTestCase	UTSwClient.cpp 14
PASS	Executing test function testMainMenuPage	UTSwClient.cpp 30
PASS	Executing test function testNewGamePage	UTSwClient.cpp 38
PASS	Executing test function testGameStartManVsAI	UTSwClient.cpp 46
PASS	Executing test function testGameStartManVsMan	UTSwClient.cpp 57
PASS	Executing test function testGameStartAIvsAI	UTSwClient.cpp 68
PASS	Executing test function testLoadGame	UTSwClient.cpp 79
PASS	Executing test function testSaveGame	UTSwClient.cpp 90
PASS	Executing test function testXWin	UTSwClient.cpp 98
PASS	Executing test function testOWin	UTSwClient.cpp 197
PASS	Executing test function testDraw	UTSwClient.cpp 311
PASS	Executing test function testConnection	UTSwClient.cpp 467
PASS	Executing test function cleanupTestCase	UTSwClient.cpp 23

Рис. 1. Список виконаних тестів

Я написав 13 тестових випадків (їх можна знайти у файлі UTSwClient.cpp), які охоплюють усі випадки можливих використання програми. Також варто зазначити, що реалізовані тестові випадки можуть містити неявні виклики методів для тестування їх функціональності. На рис. 2. зображено фрагмент коду із файлу UTSwClient.cpp, а на рис. 1. – список виконаних тестів. Як бачимо, результати виконання тестів успішні.

```

1  #include <QTest>
2  #include <QSignalSpy>
3
4  #include "../MainWindow.h"
5
6
7  class SwClientUT : public QObject
8  {
9      Q_OBJECT
10
11  private slots:
12      // Tests:
13      // GameHost::detectHostSerialPort, GameHost::isSerialPortOpen
14      void initTestCase()
15      {
16          m_magickNumber = 5;
17          m_mainWindow = new MainWindow; // Implicitly calls GameHost::detectHostSerialPort
18          QVERIFY(g_GameHost::Instance()->isSerialPortOpen());
19      }
20
21      // Tests:
22      // GameHost::closeSerialPort
23      void cleanupTestCase()
24      {
25          g_GameHost::Instance()->closeSerialPort();
26      }
27
28      // Tests:
29      // MainWindow::onMainMenu
30      void testMainMenuPage()
31      {
32          m_mainWindow->onMainMenu();
33          QCOMPARE(m_mainWindow->currentPageIndex(), 0);
34      }
35
36      // Tests:
37      // MainWindow::onNewBtn
38      void testNewGamePage()
39      {
40          m_mainWindow->onNewBtn();
41          QCOMPARE(m_mainWindow->currentPageIndex(), 1);
42      }
43
44      // Tests:
45      // MainWindow::onManVsAIBtn, MainWindow::onGameStart, GameHost::reqNewGame, GameHost::sigGameStart
46      void testGameStartManVsAI()
47      {
48          m_mainWindow->onManVsAIBtn();
49          QSignalSpy signalSpy(g_GameHost::Instance(), &GameHost::sigGameStart); // Emits MainWindow::onGameStart
50          QVERIFY(signalSpy.wait(3000));
51          QCOMPARE(signalSpy.count(), 1);
52          QCOMPARE(m_mainWindow->currentPageIndex(), 2);
53      }

```

Рис. 2. Фрагмент коду із файла UTSwClient.cpp

У моєму випадку зручно оцінювати покриття визначенням кількості виконуваних методів (слотів та сигналів в тому числі) під час тестування. Платформа тестування Qt не підтримує автоматичне вимірювання покриття, але ми можемо зробити це за допомогою нескладного аналізу.

Отже, проект складається із 4 класів (MainWindow, GameHost, GameBoard, GameButton). Нижче (у табл. 1) можна побачити загальну кількість методів, слотів і сигналів у кожному класі.

Табл. 1. Список методів, сигналів та слотів у проекті за класами

MainWindow	1 метод, 10 слотів
GameHost	9 методів, 1 слот, 5 сигналів
GameBoard	3 методи, 1 слот
GameButton	4 методи, 1 сигнал

Загалом – 35.

Мої тестові випадки покривають усі методи (слоти та сигнали в тому числі), окрім наведених у табл. 2.

Табл. 2. Непокриті методи, слоти та сигнали

Методи
<code>void GameBoard::reset()</code> <code>void GameBoard::load(const QString &cells)</code> <code>void GameButton::setState(char state)</code> <code>void GameButton::id()</code>
Слоти
<code>void GameBoard::onPlayerChoice(int cellId, signed char cellState)</code>
Сигнали
<code>void GameButton::hit(int id)</code>

Загалом непокрито – 6.

Проведемо підрахунок покриття (див. рис. 3).

```
Total = 35
Uncovered = 6
Covered = 35 - 6 = 29
Coverage = 100 * (1 - 6 / 35) = 83%
```

Рис. 2. Підрахунок покриття

За підрахунками покриття становить 83%.

Висновок

Виконуючи лабораторну роботу, я створив набір автоматизованих тестів із використанням фреймворка Qt Test. Зокрема, реалізував 13 тестових випадків (їх можна знайти у файлі UTSwClient.cpp), які охоплюють усі можливі випадки використання програми. Також підрахував загальне покриття тестами. Воно становить 83%.